

Squad Authentication & Authorization Integration Guide

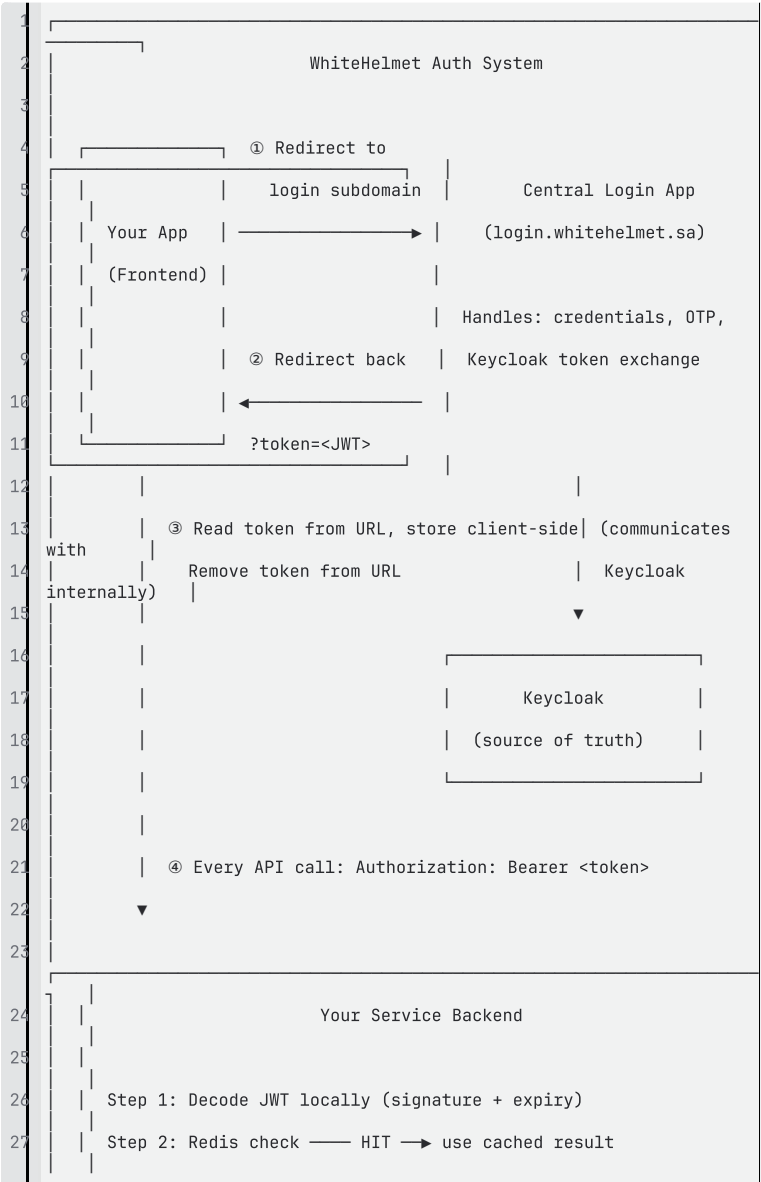
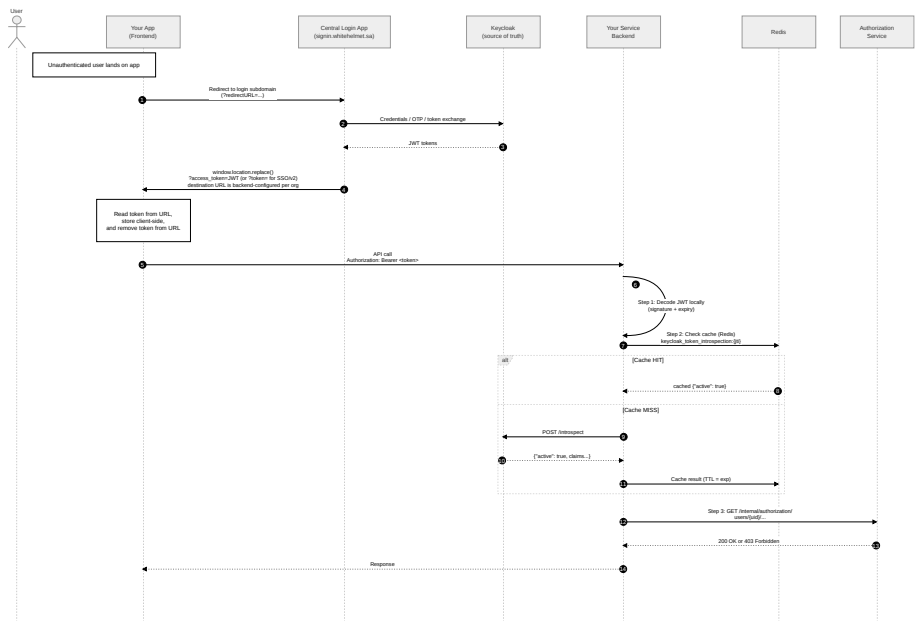
Audience: Any development squad building or integrating a new service into the WhiteHelmet platform.

Scope: How to receive a session token from the central Login App, handle logout, validate tokens (Redis caching + Keycloak introspection), and check permissions via the Centralized Authorization Service (service-to-service).

Table of Contents

- 1. Architecture Overview
 - 2. Environments & Base URLs
 - 3. Authentication — How Your App Receives the Token
 - 3.1 The Central Login App
 - 3.2 Receiving the Token After Redirect
 - 3.3 Token Storage
 - 3.4 Token Refresh
 - 3.5 Unauthenticated Users — Redirect to Login App
 - 4. Logout
 - 4.1 Single Session Logout
 - 5. Token Validation (Backend)
 - 5.1 Full Validation Flow
 - 5.2 Redis Cache Layer
 - 5.3 Keycloak Introspection Endpoint
 - 5.4 Cache Invalidation on Logout
 - 6. Authorization — Checking Permissions
 - 6.1 Check Project Access
 - 6.2 Check a Specific Permission on a Project
 - 6.3 Check a System-Level Permission
 - 6.4 List User Projects
 - 6.5 Annotate Projects (type + permissions)
 - 6.6 Batch Permissions for Multiple Projects
 - 7. Keycloak Client Setup Checklist
 - Authorization Service Errors
 - 9. FAQ
-

1. Architecture Overview





Key facts:

Component	Role	Notes
Central Login App	Handles all authentication UI: credentials, OTP, Keycloak token exchange	The frontend side needs to redirect to the login subdomain and receive the token back
Keycloak	Issues and introspects JWT tokens. Single source of truth for identity	
Redis	Caches Keycloak introspection results, shared across all services	Configure connection (The DevOps will do that)
Authorization Service	Answers permission questions via S2S HTTP calls	Call from your backend on every permission-gated request

Important: Keycloak is the source of truth. Redis is a performance cache only.

2. Environments & Base URLs

Environment	Keycloak Base URL	Realm
Development	http://authorization-microservice.stg.svc.cluster.local/api/	dev
Staging	(contact platform team)	staging
Pre-production	(contact platform team)	preprod

Production	(contact platform team)	production
-------------------	-------------------------	------------

Authorization Service base path (all environments):

```
1 | /internal/authorization/...
```

Contact the platform team (@MarioNassef / @Mohamed Diao) to get the `BASE_URL` and `client_secret` for your service in each environment.

3. Authentication — How Your App Receives the Token

Your squad does not implement login. Authentication (credentials, OTP, Keycloak token exchange) is fully handled by the central Login App. Your only responsibility is to redirect unauthenticated users to it and read the token from the URL when they return.

3.1 The Central Login App

The Login App lives on a dedicated subdomain (e.g., `login-stg.whitehelmet.sa`) and is shared across all products. It handles:

- Username + password entry
- OTP challenge (SMS / EMAIL / BOTH) when enabled for the user
- Keycloak token exchange
- Redirect back to the originating app with the issued JWT

You never need to know the internals of how login works — the Login App owns that entirely.

environment	URL
Dev	N/A
staging	login-stg.whitehelmet.sa
production	signin.whitehelmet.sa

3.2 Receiving the Token After Redirect

After successful authentication, the Login App redirects the user back to your app with the JWT access token appended as a URL query parameter:

```
1 | https://your-app.whitehelmet.sa/some-path?token=eyJhbGci...
```

What your frontend must do on landing:

1. Read the `token` query parameter from the URL.
2. Validate it is present and non-empty.
3. Store it in client-side storage (see [Section 3.3](#)).
4. **Remove the token from the URL immediately** — replace the browser history entry so the token is not visible in the address bar, bookmarks, or server logs.
5. Continue loading the app normally.

Example (Vanilla JS)

```
1 | const params = new URLSearchParams(window.location.search);
2 | const token = params.get('token');
3 |
4 | if (token) {
5 |     storeToken(token);
6 | }
```

```

6
7 // Remove token from URL
8 params.delete('token');
9 const newUrl = [
10   window.location.pathname,
11   params.toString() ? '?' + params.toString() : ''
12 ].join('');
13 window.history.replaceState({}, '', newUrl);
14 }

```

Why remove the token from the URL? Tokens in URLs can leak into browser history, server access logs, referrer headers, and analytics tools. Always strip it immediately after reading.

3.3 Token Storage

Store both the `access_token` and `refresh_token` in a way that balances security and usability for your app type:

App Type	Recommended Storage	Notes
Web (SPA, persistent session)	<code>localStorage</code> / <code>sessionStorage</code>	N/A

3.4 Token Refresh

The `access_token` has a short lifetime (default: **5 to hour minutes**). Use the `refresh_token` (default lifetime: **30 minutes**) to silently obtain a new access token without sending the user back to the Login App.

```

1 POST {KEYCLOAK_BASE_URL}/realms/{realm}/protocol/openid-connect/token
2 Content-Type: application/x-www-form-urlencoded

```

Parameter	Value
<code>client_id</code>	Your service's frontend client ID
<code>grant_type</code>	<code>refresh_token</code>
<code>refresh_token</code>	The stored refresh token

Success response — a new `access_token` and a new `refresh_token`. Replace both in storage.

Recommended refresh strategy:

```

1 1. On every API call, check if access_token expires in < 60 seconds
2 2. If yes → silently refresh first, then make the API call
3 3. On any 401 response from your API → attempt one refresh and retry
4 4. If refresh fails (refresh token expired or revoked) → clear tokens
5 and redirect user to the Login App

```

Contact the platform team for the Keycloak base URL and your frontend `client_id` per environment.

3.5 Unauthenticated Users — Redirect to Login App

When your app detects there is no valid token (on load, after refresh failure, or on 401), redirect the user to the Login App.

```

1 https://login-stg.whitehelmet.sa

```

The Login App will append `?token=<JWT>` to your app url upon success.

4. Logout

4.1 Single Session Logout

Revokes a specific token. Call the below **public endpoint** when a user needs to logout from current device/session.

1 POST {KEYCLOAK_BASE_URL}/realms/{realm}/protocol/openid-connect/revoke

2 Content-Type: application/x-www-form-urlencoded

Parameter	Value
client_id	Your service client ID
token	The access token or refresh token to revoke

After calling this endpoint, your backend must also invalidate the Redis cache for this token:

1 DEL whitehelmet_database_keycloak_token_introspection:{jti}

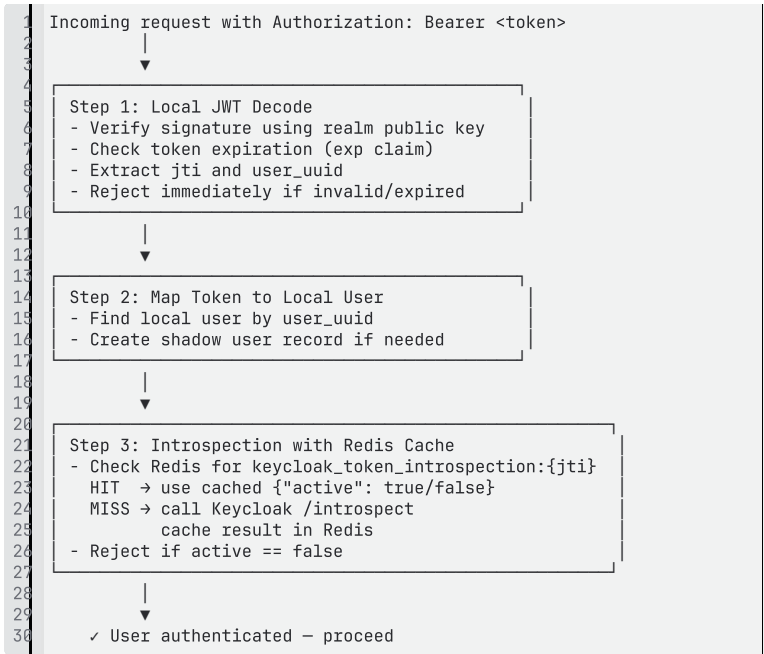
Where `jti` is the `jti` claim extracted from the JWT payload.

See Section 5.4 for the full Redis invalidation procedure.

5. Token Validation (Backend)

Every authenticated request to your service **must** validate the Bearer token. Follow this three-step flow.

5.1 Full Validation Flow



5.2 Redis Cache Layer

Redis configuration variables

Variable	Description	Example
REDIS_HOST	Redis server hostname	localhost

REDIS_PASSWORD	Redis password (null if none)	null
REDIS_PORT	Redis port	6379
REDIS_DB	Database index for token cache	8
REDIS_PREFIX	Key prefix	whitehelmet_datab ase_

Redis key structure

Key (logical)	Actual key in Redis	Type	Value	TTL
keycloak_ token_int rospection: {jti}	whitehelm et_databa se_keyclo ak_token_ introspec tion: {jti}	String	{"active" : true} (JSON)	Based on token exp
keycloak_ user_toke ns: {user_uui d}	whitehelm et_databa se_keyclo ak_user_t okens: {user_uui d}	Set (jti values)	Set of jti strings	24 hours

Always store the introspection result as a raw JSON string, not serialized objects or other encodings. This ensures cross-service compatibility (PHP, Go, Node.js, etc. can all read the same entry).

TTL calculation

```
1 | ttl_seconds = max(token.exp - now(), 5 * 60) // minimum 5 minutes
```

If the token has no exp claim, use a default TTL of **60 minutes**.

Cache lookup pseudocode

```
1 function validateToken(accessToken):
2   // Step 1: decode locally
3   claims = decodeJWT(accessToken)
4   if claims.exp < now():
5     return UNAUTHORIZED
6
7   // Step 2: check Redis
8   cached = redis.get("keycloak_token_introspection:" + claims.jti)
9   if cached != null:
10    result = json.parse(cached)
11    return result.active ? OK : UNAUTHORIZED
12
13   // Step 3: call Keycloak
14   result = keycloak.introspect(accessToken)
15   ttl = max(claims.exp - now(), 300)
16   redis.setex("keycloak_token_introspection:" + claims.jti, ttl,
17     json.encode(result))
18
19   // Track token for this user (enables logout-everywhere support)
20   redis.sadd("keycloak_user_tokens:" + claims.user_uuid, claims.jti)
```

```
20 redis.expire("keycloak_user_tokens:" + claims.user_uuid, 86400)
21
22 return result.active ? OK : UNAUTHORIZED
```

5.3 Keycloak Introspection Endpoint

```
1 POST {KEYCLOAK_BASE_URL}/realms/{realm}/protocol/openid-
  connect/token/introspect
2 Content-Type: application/x-www-form-urlencoded
```

Parameter	Value
client_id	Your service client ID
client_secret	Your service client secret
token	The access token to validate

Active token response (200 OK)

```
1 {
2   "active": true,
3   "exp": 1785318606,
4   "iat": 1753782606,
5   "jti": "onrtro:b636a440-d7e5-45ea-b41b-ddd64a58688b",
6   "iss": "https://dev-keycloak.whitehelmet.sa/realms/dev",
7   "sub": "c3482cd3-7019-4fb2-ad23-3524ab65c0a5",
8   "typ": "Bearer",
9   "azp": "your-service-be",
10  "user_uuid": "c3482cd3-...",
11  "user_id": "721",
12  "email": "user@example.com",
13  "name": "John Doe",
14  "preferred_username": "johndoe"
15 }
```

Inactive / revoked token response (200 OK)

```
1 { "active": false }
```

The HTTP status is always **200**. Use the **"active"** field to determine validity.

5.4 Cache Invalidation on Logout

Single session logout

```
1 DEL whitehelmet_database_keycloak_token_introspection:{jti}
```

Logout from all devices

```
1 jtiSet = SMEMBERS whitehelmet_database_keycloak_user_tokens:{user_uuid}
2 for each jti in jtiSet:
3   DEL whitehelmet_database_keycloak_token_introspection:{jti}
4 DEL whitehelmet_database_keycloak_user_tokens:{user_uuid}
```

6. Authorization — Checking Permissions

Once a user is authenticated (valid token), use the **Centralized Authorization Service** to check what they can do. This is a **service-to-service (S2S)** call — your backend calls the authorization service, never the frontend.

All requests require:

```
1 Authorization: Bearer <service_token>
```

Standard success envelope:

```
1 {
2   "status": "success",
3   "message": "Success",
```



```
4  "error_code": null,  
5  "meta_data": null,  
6  "errors": null,  
7  "data": [...]  
8  }
```

Standard forbidden response (403):

```
1  {  
2    "status": "error",  
3    "message": "This action is unauthorized.",  
4    "error_code": null,  
5    "data": null,  
6    "errors": ["This action is unauthorized."]  
7  }
```

6.1 Check Project Access

"Can user U access project P at all?"

```
1  GET  
2  /internal/authorization/users/{user_id}/projects/{project_id}/access  
   Authorization: Bearer <token>
```

Returns `200` with `data` payload if the user has any access, `403` if not.

Example

```
1  curl -H "Authorization: Bearer $TOKEN" \  
2  "$AUTH_SERVICE/internal/authorization/users/721/projects/99/access"
```

6.2 Check a Specific Permission on a Project

"Does user U have permission X on project P?"

```
1  GET  
2  /internal/authorization/users/{user_id}/projects/{project_id}/permissions/{permission}  
   Authorization: Bearer <token>
```

Example

```
1  curl -H "Authorization: Bearer $TOKEN" \  
2  "$AUTH_SERVICE/internal/authorization/users/721/projects/99/permissions/  
   /view_reports"
```

6.3 Check a System-Level Permission

"Does user U have global/system permission X (not scoped to any project)?"

```
1  GET /internal/authorization/users/{user_id}/permissions/{permission}  
2  Authorization: Bearer <token>
```

Example

```
1  curl -H "Authorization: Bearer $TOKEN" \  
2  "$AUTH_SERVICE/internal/authorization/users/721/permissions/manage_bill  
   ing"
```

6.4 List User Projects

"Which project IDs can user U see?"

```
1  GET /internal/authorization/users/{user_id}/projects  
2  Authorization: Bearer <token>
```

Success response `data` field:

```
1 [101, 102, 105, 210]
```

6.5 Annotate Projects (type + permissions)

"For these specific projects, what type are they and what permissions does user U have on each?"

```
1 GET /internal/authorization/users/{user_id}/projects/annotate?
  project_ids[]=101&project_ids[]=102
2 Authorization: Bearer <token>
```

Success response data field:

```
1 [
2   {
3     "project_id": 101,
4     "project_type": "construction",
5     "item_style": "standard",
6     "permissions": ["view_reports", "submit_forms"]
7   },
8   {
9     "project_id": 102,
10    "project_type": "infrastructure",
11    "item_style": "extended",
12    "permissions": ["view_reports"]
13  }
14 ]
```

This is the most efficient call when your UI needs to render menus or filter features per project — fetch the annotation once and gate all features based on it.

6.6 Batch Permissions for Multiple Projects

"For a list of project IDs, what permissions does user U have on each?"

```
1 POST /internal/authorization/users/{user_id}/projects/permissions
2 Authorization: Bearer <token>
3 Content-Type: application/json
```

```
1 {
2   "project_ids": [101, 102, 105]
3 }
```

Success response data field:

```
1 {
2   "101": ["view_reports", "submit_forms"],
3   "102": ["view_reports"],
4   "105": []
5 }
```

7. Keycloak Client Setup Checklist

When your squad registers a new service with the platform team, provide the following:

For every service (backend):

- **Service name** — human-readable name (e.g., "DMS Service")
- **Client ID** — unique identifier for your backend (e.g., `core-be`)
- **Which realms** — `dev` / `staging` / `preprod` / `production` (specify which you need)
- **Token lifespan** — default is 5 min access / 30 min refresh; request a change if needed

For frontend / mobile clients:

- **Frontend Client ID** — (e.g., `core-fe`)
- **Allowed redirect URIs** — the URLs your app will land on after login (e.g., `https://my-app.whitehelmet.sa/ *`). The Login App enforces an allowlist; unregistered URIs will be rejected.

Once registered, the platform team provides:

- `client_id` (frontend and/or backend)
- `client_secret` (backend/confidential clients only — never expose this on the frontend)
- Keycloak base URL per environment
- Confirmation that your redirect URIs are registered with the Login App

Authorization Service Errors

HTTP Status	Meaning
200	Access granted or data returned
401	Missing or invalid service bearer token
403	User is authenticated but does not have the requested permission

9. FAQ

Q: Do we need to build a login page?

A: No. The central Login App handles everything. Your only job is to redirect unauthenticated users to it (with a `redirect_uri`) and read the token from the URL when they come back.

Q: What if the user arrives without a `?token` parameter and there's no stored token?

A: Redirect them to the Login App with your app's URL as `redirect_uri`. Do not show a blank screen or an error — the redirect is the correct response for unauthenticated users.

Q: The `access_token` expired mid-session. What should the user experience?

A: The user should not notice. Your app should proactively refresh the token using the `refresh_token` before it expires (check with < 60 seconds remaining). If the refresh itself fails (refresh token expired), redirect transparently to the Login App — the user will be re-authenticated and returned to your app automatically.

Q: Do I need to call both Redis and the Authorization Service on every request?

A: Yes, but they serve different purposes. Redis + Keycloak introspection answers "is this token valid and who is the user?" The Authorization Service answers "what can this user do?" Both must pass before serving a protected resource.

Q: My service doesn't have a user table. How do I map the token to a user?

A: Use the claims in the introspection response directly (`user_id`, `user_uuid`, `email`). If you need a shadow user record for foreign keys, create it lazily on the first authenticated request. The `user_uuid` is the stable cross-service identifier.

Q: Who manages the Redis instance? Do I need to set up my own?

A: Redis is a shared platform-managed instance. Use `REDIS_DB=8` with the `whitehelmet_database_` prefix. Contact the platform team for connection credentials per environment.

Q: Our service is sold standalone. Do we still need the Authorization Service?

A: Only if your service enforces permissions scoped to platform projects or clients. If you manage your own permission model independently, you can skip Authorization Service calls. However, you must still validate tokens via Redis + Keycloak introspection — authentication is never optional.

Q: How do I handle the Authorization Service being temporarily unavailable?

A: Fail closed — return `503 Service Unavailable` rather than granting access. Never degrade to "allow all" when an auth system is unreachable. Log the failure with the request trace ID.

Q: Can my frontend call the Authorization Service directly?

A: No. The Authorization Service is internal (`/internal/...`) and accepts service-to-service bearer tokens, not user tokens. Your backend must proxy all permission checks.